CHAPTER 13

# TCP Flow & Congestion Control

rwnd · cwnd · slow start · congestion avoidance · fast retransmit · Tahoe vs Reno · throughput

**QUICK TRICK**

Sender may have in flight at most **min(rwnd, cwnd)**. Slow start: cwnd **doubles per RTT** up to ssthresh, then **+1 MSS per RTT**. Timeout: **ssthresh = cwnd/2, cwnd = 1**. 3 dup ACKs (Reno): **ssthresh = cwnd/2, cwnd = ssthresh**. Throughput ≈ **W / RTT**.

## 13.1 Flow control: protect the receiver

● **LEVEL 1 · BEGINNER**

**The problem.** A fast sender can overflow a slow receiver's buffer. **The idea.** The receiver advertises how much free buffer it has in the **window** field.

- **rwnd** = RcvBuffer − (LastByteRcvd − LastByteRead). The sender keeps **LastByteSent − LastByteAcked ≤ rwnd**.
- If rwnd = 0 the sender stops and sends small **window probes** (persist timer), so a lost window update cannot cause a deadlock.
- Silly-window syndrome (tiny windows): the sender waits to fill a segment (Nagle); the receiver advertises only when it has a decent amount free (Clark).
- Max throughput limited by the window: **rate ≤ W / RTT**. With a 16-bit window (65535 B) this caps the rate unless window scaling is used.

## 13.2 Congestion control: protect the network

● **LEVEL 2 · EXAM**

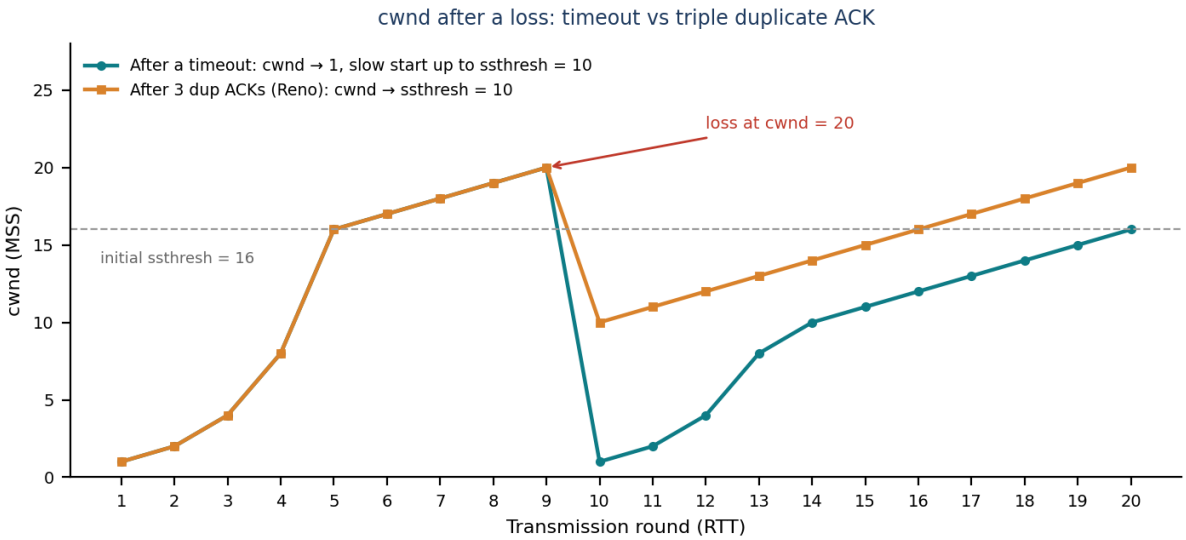
**The problem.** Too many senders overload routers; queues overflow; packets are lost. **The idea.** Each sender keeps a **congestion window cwnd** and adjusts it by what it observes: increase slowly while all is well, cut sharply when loss is seen (**AIMD**: additive increase, multiplicative decrease).

**Effective window = min(rwnd, cwnd).** Standard exam model: cwnd starts at **1 MSS**; ssthresh is given (or very large).

| Phase | Rule | Growth per RTT |
|---|---|---|
| Slow start (cwnd < ssthresh) | cwnd += 1 MSS for **each ACK** | doubles: 1, 2, 4, 8 … |
| Congestion avoidance (cwnd ≥ ssthresh) | cwnd += MSS × MSS / cwnd per ACK | **+1 MSS** |
| Timeout (loss) | ssthresh = cwnd / 2; cwnd = 1 MSS; go to slow start | – |
| 3 duplicate ACKs, Tahoe | ssthresh = cwnd / 2; cwnd = 1 MSS; slow start | – |
| 3 duplicate ACKs, Reno (fast retransmit + fast recovery) | Resend the missing segment at once; ssthresh = cwnd / 2; cwnd = ssthresh; congestion avoidance | – |

**QUICK TRICK**

When doubling would jump past ssthresh, the exam convention is: **cwnd = ssthresh** for that round, then +1 per round. (Real Reno briefly adds 3 MSS for the dup ACKs, then deflates to ssthresh.)




**Reading the picture** (initial ssthresh 16). Rounds 1–5: 1, 2, 4, 8, 16 (slow start). Then +1 per round: 17, 18, 19, 20. Loss at cwnd = 20 → ssthresh = **10**. After a **timeout** cwnd falls to 1 and the sequence is 1, 2, 4, 8, **10**, 11, 12 ...; after **3 dup ACKs in Reno** it drops only to 10 and continues 10, 11, 12 ....

## 13.3 Counting rounds and segments

● **LEVEL 3 · GATE**

- In slow start the number of segments sent in the first n rounds = $1 + 2 + 4 + \ldots + 2^{n-1} = \mathbf{2^n - 1}$.
- To send 100 segments in slow start: 1+2+4+8+16+32 = 63, adding 64 gives 127, so **7 rounds**.
- Rounds to reach cwnd = W from 1 in slow start = $\log_2 W$ (e.g. 16 needs 4 doublings).

### Average throughput of the sawtooth

If cwnd oscillates between W/2 and W, the average window is 0.75 W, so **average throughput ≈ 0.75 × W × MSS / RTT**. Another standard result (Kurose): throughput ≈ **1.22 × MSS / (RTT × √L)** for loss rate L.

## 13.4 Utilisation and window size

● **LEVEL 2 · EXAM**

To fill a link the window must be at least the **BDP** = rate × RTT (Chapter 3). Example: 100 Mbps, RTT 40 ms → BDP = $4 \times 10^6$ bits = **500,000 B**, more than 65,535 B, so **window scaling** is required.

## 13.5 Common mistakes

**COMMON MISTAKE**

- Resetting cwnd to 1 after 3 duplicate ACKs in Reno (it becomes the new ssthresh).
- Taking ssthresh = cwnd/2 from the **wrong** cwnd. Use the cwnd at the moment of loss.
- Forgetting that sending is limited by min(rwnd, cwnd), not cwnd alone.
- Letting slow start jump above ssthresh instead of capping at ssthresh.
- Mixing "per RTT" growth with "per ACK" growth.

**QUICK TRICK**

Memory line: "Timeout is serious: back to 1. Three dup ACKs is mild: halve and carry on."

## GATE QUICK CHECK — Chapter 13

```
send window = min(rwnd, cwnd)          rate <= W/RTT          BDP = rate * RTT
slow start x2 per RTT -> ssthresh -> +1 MSS per RTT
timeout    : ssthresh = cwnd/2 ; cwnd = 1
3 dup ACKs : Tahoe cwnd = 1 ; Reno cwnd = ssthresh = cwnd/2
segments in n slow-start rounds = 2^n - 1       sawtooth avg = 0.75 W / RTT
```

## Practice questions

● **LEVEL 1 · BEGINNER**

**Q1.** If rwnd = 8 KB and cwnd = 12 KB, how much can the sender have in flight?

**Q2.** A timeout happens at cwnd = 20 MSS. New ssthresh and cwnd?

**Q3.** Three duplicate ACKs at cwnd = 20. New cwnd under Tahoe and Reno?

**Q4.** Which mechanism protects the receiver and which protects the network?

● **LEVEL 2 · EXAM**

**Q5.** cwnd starts at 1 MSS and ssthresh = 8. List cwnd for rounds 1 to 8.

**Q6.** Slow start from 1 MSS: how many rounds to reach cwnd = 16, and how many segments sent in those rounds?

**Q7.** Maximum rate with a 65,535 B window and RTT 50 ms.

**Q8.** Window to fill 100 Mbps with RTT 40 ms?

● **LEVEL 3 · GATE**

**Q9.** ssthresh = 8 initially, cwnd = 1. Timeout occurs when cwnd = 10 (round 6). List cwnd for rounds 1–11.

**Q10.** A TCP sawtooth peaks at W = 20 segments of 1000 B with RTT 100 ms. Average throughput?

**Q11.** How many rounds of slow start are needed to send 100 segments, starting from cwnd = 1 (no ssthresh limit)?

**Q12.** Why does a zero window need a persist timer?

## Answers

| Q | Solution |
|---|---|
| 1 | min = **8 KB**. |
| 2 | ssthresh = **10**, cwnd = **1** MSS. |
| 3 | Tahoe: **1**. Reno: **10** (= new ssthresh). |
| 4 | Receiver: **flow control (rwnd)**. Network: **congestion control (cwnd)**. |
| 5 | **1, 2, 4, 8, 9, 10, 11, 12**. |
| 6 | cwnd is 16 at round 5, i.e. after **4 doublings** (1→2→4→8→16). Segments sent in the first 4 rounds: 1 + 2 + 4 + 8 = **15** ($2^4 - 1$). |
| 7 | 65535 × 8 / 0.05 = **10.49 Mbps**. |
| 8 | BDP = $10^8$ × 0.04 = $4 \times 10^6$ bits = **500,000 B**. |
| 9 | Rounds 1–6: 1, 2, 4, 8, 9, 10. Timeout at round 6: ssthresh = 5, cwnd = 1. Rounds 7–11: **1, 2, 4, 5, 6**. Full list: 1, 2, 4, 8, 9, 10, 1, 2, 4, 5, 6. |
| 10 | 0.75 × 20 × 8000 / 0.1 = **1.2 Mbps**. |
| 11 | 1+2+4+8+16+32 = 63 (6 rounds) is not enough; 7 rounds give 127. **7 rounds**. |
| 12 | If the receiver's window-update ACK is lost, both sides would wait forever. The sender's probe forces a fresh window advertisement. |

**Next:** Chapter 14 — Socket API and UDP.